Mathematical Formulation of the Porous Navier-Stokes Solvers

Antigravity AI Project Authors

April 14, 2026

1 Introduction

This document outlines the formulation of the Orthogonal Sub-Grid Scales (OSGS) stabilization method and the nonlinear globalization strategy for the generalized porous Navier–Stokes equations. The algorithmic structure strictly separates the continuous finite element operator assembly from the discrete algebraic safeguarding mechanisms.

2 Solver Parameters

All bounds mapping to the internal runtime configuration are classified below.

Table 1: Nonlinear and Staggered Solver Configuration Parameters

Symbol	**Code Property**	**Description**
N_{OSGS}	`stab_cfg.osgs_iterations`	Maximum number of outer staggered loops permitted.
N_{inner}	`stab_cfg.osgs_inner_newton_iters`	Iteration limit for each inner monolithic Newton-Raphson solve.
m_{warmup}	`stab_cfg.osgs_warmup_iterations`	Transition frame halting OSGS decoupled initialization.
N_{inc}	`sol_cfg.max_increases`	Allowable successive iterations where the scalar merit function expands.
N_{plateau}	`mms_cfg.require_consecutive_passes`	Number of consecutive error verifications achieving steady reduction.
$\alpha_{\min}$	`sol_cfg.linesearch_alpha_min`	Minimum allowable geometric step size.
c_1	`sol_cfg.armijo_c1`	Armijo condition parameter evaluating sufficient residual descent.
F_{div}	`sol_cfg.divergence_merit_factor`	Expansion threshold evaluating catastrophic divergence bounds.
τ_{OSGS}	`stab_cfg.osgs_tolerance`	Tolerance evaluating acceptable outer state continuity.
τ_{proj}	`stab_cfg.osgs_projection_tol`	Tolerance evaluating sub-grid spatial projection limits.
τ_{warmup}	`stab_cfg.osgs_warmup_tolerance`	Relaxation floor bounding initial uncoupled OSGS inner iterations.
τ_{ftol}	`sol_cfg.ftol`	Terminal nonlinear residual tolerance bounding the sequence ℓ^∞ norm.
τ_{xtol}	`sol_cfg.xtol`	Coordinate step norm limit halting sequence updates.
$\tau_{\text{stagnation}}$	`sol_cfg.stagnation_noise_floor`	Noise-floor limit preventing spurious machine-epsilon stalls.
τ_{error}	`mms_cfg.tau_err`	Plateau relative error bound verifying MMS limits.

3 Algorithmic Abstractions and Code Equivalence

To bridge the gap between pure mathematical representations and the programmatic implementation, we rigorously define the equivalence between the algorithm functions presented in this section and their corresponding typed Julia structures.

Table 2: Equivalence Between Mathematical Functions and Code Subroutines

Mathematical Function	**Code Signature Equivalent**
ExactNewtonPipeline($\boldsymbol{x}^0$)	`Gridap.Algebra.solve!(x, solver, op_newton_init)`
SolveLinearSystem($\mathcal{J}_i, \boldsymbol{b}_i$)	`eval_linear_system_resolution!(dx, A, b, solver, cache)`
ArmijoLinesearch(...)	`eval_armijo_linesearch_pass!(x, x_old, b, dx, op, dir_deriv, solver, w, state)`
EvaluateSafeguards(...)	`eval_safeguard_termination_bounds!(solver, state)`
PicardFallback(...)	Iterative mapping fallback executing `solve!(x, solver_picard, op_picard)` within `solve_system()`
OSGSFractionalRelaxation(...)	Outer fixed-point evaluation recursively yielding converged OSGS projections terminal to `solve_system()`
VerifyMMSPlateau(U_h^m, U_{ex})	Explicit residual error plateau verification intercept evaluated geometrically via `mms_cfg.oracle()`

Let Θ_{params} denote the grouped parameters for nonlinear iteration constraints, scalar tolerances, and bounds. When algorithms dynamically query these structures, they formally extract from:

$$\Theta_{\text{params}} = \{N_{\text{inner}}, N_{\text{inc}}, N_{\text{ls_max}}, \tau_{\text{ftol}}, \tau_{\text{xtol}}, \tau_{\text{stagnation}}, \alpha_{\min}, F_{\text{div}}\} \tag{1}$$

4 Algorithmic Structure

Let the primary velocity-pressure state be $U_h = (\boldsymbol{u}_h, p_h) \in \mathcal{X}_h$. The continuous sub-grid projection variables are $\boldsymbol{\pi}_h = (\boldsymbol{\pi}_{h,u}, \pi_{h,p})^\top \in \mathcal{X}_h$.

To natively balance the numerical disparity between the momentum and mass conservation equations without *a priori* macroscopic scaling, we extract an element-wise scaling vector from the exact Jacobian at each iteration: $w_k = \max(|[\mathcal{J}]_{kk}|, \epsilon_{\text{mach}}\|\text{diag}(\mathcal{J})\|_\infty, \epsilon_{\text{mach}})$. The diagonally equilibrated dimensionless merit function is defined as:

$$\Phi_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{2}\sum_{k=1}^{N}\left(\frac{b_k(\boldsymbol{x})}{w_k}\right)^2 \tag{2}$$

Algorithm 1: Algorithm A: ExactNewtonPipeline

Numerical Parameters: $\tau_{\text{ftol}}, N_{\text{inner}}, N_{\text{inc}}, \tau_{\text{stagnation}}, \tau_{\text{xtol}}$
Input: Initial geometric fields $\boldsymbol{x}^0 \in \mathcal{X}_h$
Output: $(S_{\text{success}} \in \{\text{True}, \text{False}\},\ I_{\text{iters}} \in \mathbb{N},\ \|\boldsymbol{b}\|_\infty \in \mathbb{R})$

Initialize $\boldsymbol{b}_{\text{best}} = \|\boldsymbol{b}(\boldsymbol{x}^0)\|_\infty, I_{\text{up}} = 0$;
if $\|\boldsymbol{b}(\boldsymbol{x}^0)\|_\infty \leq \tau_{ftol}$ **then**
  return Success (Exit: Initial F-Tol)
end
for $i = 1, \ldots, N_{inner}$ **do**
  Compute continuous variables: $\mathcal{J}_i, \boldsymbol{b}_i, \boldsymbol{w}, \Phi_{\text{curr}}$;
  $S_{\text{fail}}, \delta\boldsymbol{x} \leftarrow \text{SolveLinearSystem}(\mathcal{J}_i, \boldsymbol{b}_i)$;
  if S_{fail} **then**
    Restore $\boldsymbol{x} \leftarrow \boldsymbol{x}_{\text{best}}$. **return** $(S_{\text{conv}} = \text{False}, I_{\text{iters}} = i, \|\boldsymbol{b}_{\text{best}}\|_\infty)$;
  end
  $S_{\text{step}}, \alpha, L_{\text{step}}, \hat{\boldsymbol{b}}, \hat{\Phi} \leftarrow \text{ArmijoLinesearch}(\boldsymbol{x}, \boldsymbol{x}_{\text{old}}, \boldsymbol{b}, \delta\boldsymbol{x}, \Phi_{\text{curr}})$;
  if not S_{step} **or** $\|\hat{\boldsymbol{b}}\|_\infty < \boldsymbol{b}_{best}$ **then**
    $\boldsymbol{x}_{\text{best}} = \boldsymbol{x}, \boldsymbol{b}_{\text{best}} = \|\hat{\boldsymbol{b}}\|_\infty$;
  end
  $R_{\text{stop}}, S_{\text{conv}}, S_{\text{break}}, I_{\text{up}} \leftarrow \text{EvaluateSafeguards}(i, S_{\text{step}}, L_{\text{step}}, \hat{\boldsymbol{b}}, \hat{\Phi}, \Phi_{\text{curr}})$;
  if S_{break} **then**
    if not S_{conv} **then**
      Restore $\boldsymbol{x} \leftarrow \boldsymbol{x}_{\text{best}}$
    end
    return $(S_{\text{conv}}, i, \|\boldsymbol{b}_{\text{best}}\|_\infty)$;
  end
end

Algorithm 2: Algorithm A.1: SolveLinearSystem

Numerical Parameters: (None)
Input: Sparse continuous Jacobian $\mathcal{J}_i$, Residual array $\boldsymbol{b}_i$
Output: $(S_{\text{fail}} \in \{\text{True}, \text{False}\},\ \delta\boldsymbol{x} \in \mathbb{R}^N)$

---

try {
  Solve linear system functionally: $\mathcal{J}_i \delta\boldsymbol{x} = \boldsymbol{b}_i$;
} **catch** {
  return $(\text{True}, \delta\boldsymbol{x}_{\text{undefined}})$; // Linear Solve Exception
}
if $\delta\boldsymbol{x}$ *contains NaNs* **then**
  return $(\text{True}, \delta\boldsymbol{x}_{\text{undefined}})$; // Catastrophic Math Failure
end
return $(\text{False}, \delta\boldsymbol{x})$; // Linear System Bounded

Algorithm 3: Algorithm A.2: ArmijoLinesearch

Numerical Parameters: $\alpha_{\min}, c_1, \gamma, N_{\text{ls_max}}$ (extracted from Θ_{params})

Input: State fields $\boldsymbol{x}, \boldsymbol{x}_{\text{old}}, \boldsymbol{b}$, step mapping $\delta\boldsymbol{x}$, exact directional derivative D, current merit Φ_{curr}

Output: $(S_{\text{accepted}} \in \{\text{True}, \text{False}\},\ \alpha \in \mathbb{R}^+,\ L_{\text{step}} = \|\alpha\delta\boldsymbol{x}\| \in \mathbb{R}^+,\ \|\hat{\boldsymbol{b}}\|_\infty \in \mathbb{R},\ \hat{\Phi}_{\boldsymbol{w}} \in \mathbb{R})$

Initialize $\alpha = 1.0, S_{\text{step_accepted}} = \text{false}$;
for $j = 1, \ldots, N_{ls_max}$ **do**
  $\boldsymbol{x}_{\text{trial}} = \boldsymbol{x}_{\text{old}} - \alpha\delta\boldsymbol{x}$;
  Compute $\hat{\boldsymbol{b}} = \boldsymbol{b}(\boldsymbol{x}_{\text{trial}})$ and $\hat{\Phi}_{\boldsymbol{w}} = \Phi_{\boldsymbol{w}}(\boldsymbol{x}_{\text{trial}})$;
  if not $isnan(\hat{\Phi}_{\boldsymbol{w}})$ **and** $\hat{\Phi}_{\boldsymbol{w}} \leq \Phi_{curr} + c_1\alpha(-2\Phi_{curr})$ **then**
    $S_{\text{step_accepted}} = \text{true}$. **break**;
  end
  $\alpha = \gamma\alpha$;
  if $\alpha \leq \alpha_{min}$ **then**
    break;
  end
end
return $(S_{\text{step_accepted}},\ \alpha,\ \|\alpha\delta\boldsymbol{x}\|,\ \|\hat{\boldsymbol{b}}\|_\infty,\ \hat{\Phi}_{\boldsymbol{w}})$;

Algorithm 4: Algorithm A.3: EvaluateSafeguards

Numerical Parameters: $\tau_{\text{ftol}}, \tau_{\text{stagnation}}, \tau_{\text{xtol}}, N_{\text{inner}}, N_{\text{inc}}, F_{\text{div}}$ (extracted from Θ_{params})

Input: Iteration i, Validation $S_{\text{step_accepted}}$, step norm L_{step}, current residual $\|\hat{\boldsymbol{b}}\|_\infty$, scalar merit $\hat{\Phi}_{\boldsymbol{w}}$, base merit Φ_{curr}

Output: $(R_{\text{stop}},\ S_{\text{conv}} \in \{\text{True}, \text{False}\},\ S_{\text{break}} \in \{\text{True}, \text{False}\},\ I_{\text{up}} \in \mathbb{N})$

```
if ||b̂||_∞ ≤ τ_ftol then
    return F-Tol Reached, true, true, I_up;
end
if not S_step_accepted then
    if ||b̂||_∞ ≤ τ_stagnation then
        return Noise Floor Stagnation, true, true, I_up;
    end
    return Linesearch Failed, false, true, I_up;
end
if Φ̂_w > Φ_curr · F_div then
    I_up = I_up + 1;
    if I_up ≥ N_inc then
        return Merit Divergence Escaped, false, true, I_up;
    end
end
else
    I_up = 0;
end
if L_step ≤ τ_xtol then
    if ||b̂||_∞ ≤ τ_stagnation then
        return Noise Floor Stagnation, true, true, I_up;
    end
    return X-Tol Stagnation, false, true, I_up;
end
if i == N_inner then
    if ||b̂||_∞ ≤ τ_stagnation then
        return Noise Floor Stagnation, true, true, I_up;
    end
    return Iteration Cap Stagnation, false, true, I_up;
end
return In-Progress, false, false, I_up;
```

Algorithm 5: Algorithm B: PicardFallback

Numerical Parameters: $\tau_{\text{ftol}}, \tau_{\text{stagnation}}$

Input: Initial field guess $\boldsymbol{x}^0 \in \mathcal{X}_h$, Preconditioner backup $\boldsymbol{x}^0_{\text{backup}} \in \mathcal{X}_h$

Output: $(S_{\text{success}} \in \{\text{True}, \text{False}\},\ I_{\text{iters}} \in \mathbb{N},\ \|\boldsymbol{b}\|_\infty \in \mathbb{R})$

```
Configure parameters zeroing sub-grid scales: π_h = 0;
Attempt Newton resolution: S_N, ··· ← ExactNewtonPipeline(x^0) targeting τ_ftol;
if S_N formally aborts (Exit via Linesearch Failed or Merit Divergence Escaped) then
    Evaluate strictly guarded Picard Homotopy pass smoothing fields towards stable continuous
      initial geometry quadratic basins;
    Re-evaluate Newton sequence definitively mapping cleanly:
      S_success, I_iters, ||b||_∞ ← ExactNewtonPipeline(x^0_smoothed);
    if Terminal evaluation fundamentally bypasses both F-Tol Reached and Noise Floor Stagnation
      structures then
        return (False, I_iters, ||b||_∞) ;          // Catastrophic Algebraic Sequence Stagnation
    end
end
return (True, I_iters, ||b||_∞) ;                   // Mathematically Locked Globally Bounded State
```

Algorithm 6: Algorithm C: OSGSFractionalRelaxation

Numerical Parameters: $N_{\text{OSGS}}, m_{\text{warmup}}, \tau_{\text{warmup}}, \tau_{\text{ftol}}, \tau_{\text{OSGS}}, \tau_{\text{proj}}, \tau_{\text{stagnation}}$
Input: Converged steady state $U_h^{\text{ASGS}} \in \mathcal{X}_h$, Algebraic forms $(\mathcal{R}_u, \mathcal{R}_p)$
Output: $(S_{\text{success}} \in \{\text{True}, \text{False}\},\ \boldsymbol{\pi}_{h,u} \in \boldsymbol{V}_h,\ \pi_{h,p} \in Q_h,\ I_{\text{iters}} \in \mathbb{N},\ \|\boldsymbol{b}\|_\infty \in \mathbb{R})$

Pre-allocate strictly invariant orthogonal mappings: $M_u \leftarrow \int V \cdot U d\Omega, M_p \leftarrow \int Q \cdot P d\Omega$;
Initialize $U_h^0 = U_h^{\text{ASGS}}$;
Bootstrap initial sub-grid projections: $\boldsymbol{\pi}_{h,u}^0 = \Pi_{\boldsymbol{V}}(\mathcal{R}_u(U_h^0))$ and $\pi_{h,p}^0 = \Pi_Q(\mathcal{R}_p(U_h^0))$;
for $m = 1, \ldots, N_{OSGS}$ **do**
  Set inner tolerance limit: $\tau_{\text{inner}}^m = (m \leq m_{\text{warmup}})\ ?\ \max(\tau_{\text{ftol}}, \tau_{\text{warmup}}) : \tau_{\text{ftol}}$;
  Set $\boldsymbol{\pi}_h = \boldsymbol{\pi}_h^{m-1}$ fixed, resolve continuous algebraic system executing strict fraction mapping via ExactNewtonPipeline targeting limited iterations N_{inner};
  if *Monolithic sweep returns algebraic collapse natively* (*Linesearch Failed* **or** *Merit Divergence Escaped*) **then**
    Abort structural iteration immediately (mathematically divergent algebraic boundaries violated);
  end
  `/* Evaluate geometric state fraction shift explicitly evaluating explicit mass vectors linearly: */`
  $d_{\text{state}}^m = \max\left(\|\boldsymbol{u}_h^m - \boldsymbol{u}_h^{m-1}\|_{L^\infty(\Omega)},\ \|p_h^m - p_h^{m-1}\|_{L^\infty(\Omega)}\right)$;
  Update continuous evaluation geometries arrays natively: $\boldsymbol{\mathcal{R}}_u^m = \mathcal{R}_u(U_h^m)$ and $\mathcal{R}_p^m = \mathcal{R}_p(U_h^m)$;
  Update continuous orthogonal projections: $(\boldsymbol{\pi}_{h,u}^m, \pi_{h,p}^m) = \left(\Pi_{\boldsymbol{V}}(\boldsymbol{\mathcal{R}}_u^m), \Pi_Q(\mathcal{R}_p^m)\right)$;
  `/* Independent Block-Weighted Anderson Acceleration */`
  (Optional) Filter limits through separated bounds using L^2 mass matrices:

$$\boldsymbol{\pi}_{h,u}^m \leftarrow \text{Anderson}(\text{History}_u, \boldsymbol{\pi}_{h,u}^{m-1}, \boldsymbol{\pi}_{h,u}^m, M_u)$$
$$\pi_{h,p}^m \leftarrow \text{Anderson}(\text{History}_p, \pi_{h,p}^{m-1}, \pi_{h,p}^m, M_p)$$

  `/* Define orthogonal projection drift explicitly evaluating algebraic block arrays: */`
  $d_\pi^m = \max\left(\sqrt{(\delta\boldsymbol{\pi}_{h,u}^m)^\top M_u \delta\boldsymbol{\pi}_{h,u}^m},\ \sqrt{(\delta\pi_{h,p}^m)^\top M_p \delta\pi_{h,p}^m}\right)$;
  if $(d_{state}^m \leq \max(\tau_{OSGS}, \tau_{stagnation}))$ **AND/OR** $(d_\pi^m \leq \max(\tau_{proj}, \tau_{stagnation}))$ **then**
    return `true`, $\boldsymbol{\pi}_{h,u}^m, \pi_{h,p}^m, I_{\text{iters}}, \|\boldsymbol{b}\|_\infty$; // Validated Staggered Boundary
  end
end
return `false`, $\boldsymbol{\pi}_{h,u}^m, \pi_{h,p}^m, I_{\text{iters}}, \|\boldsymbol{b}\|_\infty$; // Catastrophic OSGS Divergence Abort

Algorithm 7: Algorithm D: VerifyMMSPlateau

Numerical Parameters: $N_{\text{plateau}}, \tau_{\text{error}}$
Input: Candidate field state $U_h^m \in \mathcal{X}_h$, Exact analytical state $U_{\text{exact}} \in \mathcal{X}$
Output: $(S_{\text{plateau_valid}} \in \{\text{True}, \text{False}\},\ I_{\text{iters}} \in \mathbb{N},\ \|\boldsymbol{b}\|_\infty \in \mathbb{R})$

Compute continuous tracking errors via explicit integrals: $E^m_{\boldsymbol{u},L^2} = \|\boldsymbol{u}_h^m - \boldsymbol{u}_{\text{ex}}\|_{L^2(\Omega)}$, $E^m_{p,L^2} = \|p_h^m - p_{\text{ex}}\|_{L^2(\Omega)}$;
Define local error variance tracking ratios over strictly isolated elements:

$$r_{\max} = \max\left(\frac{|E^m_{\boldsymbol{u},L^2} - E^{m-1}_{\boldsymbol{u},L^2}|}{\max(E^m_{\boldsymbol{u},L^2}, E^{m-1}_{\boldsymbol{u},L^2}, \epsilon_{\text{err}})},\quad \frac{|E^m_{p,L^2} - E^{m-1}_{p,L^2}|}{\max(E^m_{p,L^2}, E^{m-1}_{p,L^2}, \epsilon_{\text{err}})}\right) \tag{3}$$

;
if $r_{max} \leq \tau_{error}$ **then**
  $I_{\text{passes}} = I_{\text{passes}} + 1$;
end
else
  $I_{\text{passes}} = 0$;
end
if $I_{passes} \geq N_{plateau}$ **then**
  return `true`, $I_{\text{iters}}, \|\boldsymbol{b}\|_\infty$; // MMS Plateau Satisfied
end
return `false`, $I_{\text{iters}}, \|\boldsymbol{b}\|_\infty$; // MMS Budget Exhausted

5 Structural Orchestration and Convergence Analysis

The solver pipeline rigorously separates pure Newton-Raphson mathematics from physical boundary analysis through a cascading fallback strategy. By explicitly monitoring algorithmic exit bounds, the system aggressively tests fast evaluations while preserving mathematical robustness across dimensionless parameter extrema (e.g., $Re, Da \sim 10^6$).

5.1 Algorithmic Fallback Progression

Robustness is achieved by evaluating nonlinear failures geometrically rather than relying exclusively on continuous float exceptions. The progression orchestrates the solvers as follows:

1. **Aggressive Exact Newton Initialization (Algorithm A):** The solver unconditionally attempts to root the initial guess directly inside the quadratic convergence basin using pure analytic derivatives. The internal logic delegates the mathematical updates to linear resolution (**Algorithm A.1**), safe bounded fractional steps (**Algorithm A.2**), and strict termination guards (**Algorithm A.3**).

2. **Picard Globalization Fallback (Algorithm B):** If **Algorithm A** aborts aggressively due to linesearch depletion evaluated via **Algorithm A.3** (Linesearch Failed) or chaotic divergence (Merit Divergence Escaped), the sequence retreats securely. It temporarily abandons exact Jacobian symmetry and invokes classical Picard iterations. Because the Picard Jacobian actively drops the advective coupling tensor, it stabilizes chaotic boundaries, smoothing the global fields strictly into the local quadratic basin before dynamically re-engaging **Algorithm A** seamlessly to securely polish the algebraic residual.

   Dynamic Adaptivity: The structural limits mapping the Picard configuration unconditionally adapt structurally evaluating the convection Reynolds bounds (Re) and reaction Damköhler extents (Da). Highly convective geometry thresholds ($Re \geq 10^4$) explicitly expand iterative smoothing limits dynamically allowing sufficient boundary layer relaxation preventing subsequent Newton convergence breakdown arrays.

5.2 Adaptive Spatial τ_{ftol} Constraint Constraints

In continuous parameter extraction runs, maintaining rigidly static algebraic stopping conditions structurally disrupts fine-mesh evaluations. The orchestration pipeline continuously scales τ_{ftol} conditionally proportionate directly against the projected analytical error variance:

$$\tau_{\text{ftol, dynamic}} = \max\left(\tau_{\text{ftol, static}},\ \min(C_{\text{ceiling}},\ C_{\text{sf}} \times h^{k+1})\right) \tag{4}$$

Where h represents the linear block resolution scalar and k represents the velocity finite element interpolant order. Natively adapting these limits using the bounding scalar C_{sf} (spatial safety factor) ensures the non-linear algebraic root is consistently evaluated proportionally beneath topological precision barriers ($C_{\text{sf}} \times h^{k+1}$). The absolute constraint ceiling C_{ceiling} intrinsically proves algorithmic execution avoids catastrophically locking the non-linear matrices natively seeking functionally impossible precision topologies bounds on ultra-coarse grid spans.

5.3 Dynamic Stagnation bounds and Mathematical Truthfulness

As established in **Algorithm A.3**, the $\tau_{\text{stagnation}}$ check functions explicitly as an absolute terminating safety net rather than an unconditional shortcut. The algebraic noise floor is strictly verified *only* if a catastrophic structural exit (Linesearch Failed, Iteration Cap) terminates the sequence gracefully prior to mathematically striking the τ_{ftol} bound identically.

False Equivalency Failsafe: It is mathematically critical to logically decouple *noise floor saturation* from *step exhaustion*. If the inner solver terminates natively via iteration budget depletion (Iteration Cap Stagnation) or extreme coordinate stalling (X-Tol Stagnation) while tracking an algebraic residual strictly above the dynamic noise floor ($\|\boldsymbol{b}\|_\infty \gg \tau_{\text{stagnation}}$), the pipeline classifies the state as an unconditional numerical failure. The orchestration framework explicitly forbids coercing unconditionally exhausted sequences into "pseudo-converged" approvals, natively protecting the staggering fractional structures from bootstrapping completely unresolved mathematical topologies.

Because classical double precision noise floors explicitly skew dynamically as fine mesh matrices scale topologically (condition parameter $K \approx \mathcal{O}(h^{-2})$), the orchestration logic natively maps dynamic extrapolation extracting algebraic condition limits bounding against target physical norms ($\tau_{\text{stagnation}} \propto 10^{-16} N_{\text{dofs}}^2 \max(1, Re)$). Natively ensuring that these bounds remain distinctly superior linearly to τ_{ftol} rigorously prevents the fine-mesh algorithms from aborting physically viable solution sequences prematurely.

5.4 Fractional Verification Requirements

Instead of implicitly treating fractional evaluation success as mathematically rigorous, **Algorithm C (OSGS)** binds independent mathematical criteria: A fractional update sweep is deemed converged exclusively when *either* or *both* the spatial $L^\infty(\Omega)$ coordinate drift (d^m_{state}) *and/or* orthogonal projection vectors (d^m_π) independently hit continuous equilibrium τ_{OSGS}. If the inner linear fractional solver mapped via **Algorithm A** collapses (Linesearch Failed), the staggered loop abruptly halts instantly recording a native fraction abort rather than falsely reporting stable limits conditionally.

5.5 OSGS Fractional Relaxation (Algorithm C) Mechanics

The Orthogonal Sub-Grid Scales (OSGS) algorithm represents the core physical stabilization technique for convection- and reaction-dominated porous flow regimes. Rather than absorbing the finite element residuals unconditionally into the unresolved sub-grid scales (as the classical ASGS method does), OSGS rigorously restricts the sub-grid scales purely to the orthogonal complement of the finite element space ($\mathcal{X}^\perp_{h0}$). Mathematical orthogonality requires computing the L^2-projection of the continuous primary residuals onto the finite element spaces, $\boldsymbol{\pi}_h = (\boldsymbol{\pi}_{h,u}, \pi_{h,p})$, and isolating it from the strong residual loop via **Algorithm C**.

5.5.1 Algorithmic Modus Operandi & Goals

To avert generating an intractable, dense, monolithic Jacobian comprising the exact Fréchet derivatives of the continuous L^2 projection operator, **Algorithm C** utilizes a staggered (fixed-point) geometric block-relaxation architecture:

1. **Implicit Monolithic Kinematic Sweep:** The primary generalized variables $U_h = (\boldsymbol{u}_h, p_h)$ are resolved natively employing **Algorithm A** (`ExactNewtonPipeline`), holding the orthogonal residual projections $\boldsymbol{\pi}_h$ temporarily frozen.

2. **Analytical Projection:** The updated robust residuals $\mathcal{R}_u(U_h)$ and $\mathcal{R}_p(U_h)$ are explicitly evaluated and projected onto $\boldsymbol{V}_h$ and Q_h. To securely execute this within sub-millisecond budgets, stationary L^2 mass matrices (M_u, M_p) are geometrically pre-factored via `LUSolver()` prior to the main sequence.

3. **Diagnostic Drift Measurement:** The structural distance evaluated across explicit consecutive steps in **Algorithm C** determines geometric convergence. Both the Cartesian state magnitude (d_{state}) and the mass-weighted sub-grid variance (d_π) are mapped out.

5.5.2 Numerical Parameter Controls

The physical staggering explicitly binds bounded geometric configuration constraints:

- $N_{\mathbf{OSGS}}$ (`osgs_iterations`): Establishes an absolute physical loop ceiling. It structurally terminates cycles averting limit-cycle bounds if iterative projection drifting never formally stagnates algebraically.

- $\tau_{\mathbf{OSGS}}$ (`osgs_tolerance`) & $\tau_{\mathbf{proj}}$: Distinguish the staggered stopping criteria from continuous inner solver algebraic saturation (τ_{ftol}). They dictate precisely when the orthogonal step geometrically locks—often several orders of magnitude above strict Newton terminal zeroes.

- $m_{\mathbf{warmup}}$ & $\tau_{\mathbf{warmup}}$: Prevent chaotic, rigid over-solving natively in early fractional limits before the frozen explicit fields settle. They forcefully cap inner Newton resolution explicitly yielding soft staggered updates first.

5.6 Block-Weighted Anderson Acceleration

In regimes of extreme stiffness, particularly at high Damköhler (Da) and Reynolds (Re) numbers, the iterative resolution of the sub-grid scales can degenerate into limit-cycle oscillations. To bypass these rigid algebraic cycles, the pipeline employs a sequence extrapolation method based on Anderson Acceleration.

Unlike global Newton accelerators, this module operates strictly on the continuous sub-grid projections $(\boldsymbol{\pi}_{h,u}, \pi_{h,p})$. To mathematically respect the spatial topology without artificially filtering distinct fields, the sequence extrapolation is executed through independent bounded blocks utilizing the pre-factored L^2 invariant mass matrices $(M_u,\ M_p)$:

1. **Block Isolation:** Velocity $(\boldsymbol{\pi}_{h,u})$ and pressure $(\pi_{h,p})$ histories are mapped and solved orthogonally averting cross-dimensional float pollution.

2. **Topology-Constrained Minimization:** The optimization explicitly minimizes the residual sequence $\min_\alpha \|R_k\alpha\|^2_{L^2(\Omega)}$ by internally leveraging the exact mass inner products $R_k^\top M R_k$ instead of ambiguous continuous vector scalar dot products.

This physical tracking ensures that the accelerated fraction optimally navigates the physical domains strictly avoiding mathematically false numerical bounds.

5.7 Method of Manufactured Solutions Verification (Algorithm D)

During convergence and formulation verification sweeps, continuous tracking algorithms are externally monitored via **Algorithm D**. The plateau methodology ensures that iterative bounds are evaluated utilizing true algebraic stability. Instead of relying solely on the terminal Newton bounds of **Algorithm A**, **Algorithm D** measures the physical L^2 continuous variance of sequential solutions dynamically mapped against analytic exact solutions (U_{exact}). This cleanly separates numerical noise-floor convergence bounds from the true continuous Galerkin sequence mappings natively.